2026 SISBID Clustering Lab

Genevera I. Allen & Yufeng Liu

Data set: Author word-count data

This data set contains word counts from chapters written by four authors. The rows are text chapters and the columns are words, with one additional identifier column in the original data.

This lab combines ideas from dimension reduction and clustering. The goal is not only to form clusters, but also to explain and visualize the structure that you find.

By the end of the lab, you should be able to:

- Cluster author texts in an unsupervised manner.
- Compare how preprocessing, distance choice, and linkage choice affect clustering results.
- Identify words that help separate the author texts.
- Visualize texts, words, and clusters in low dimensions.
- Explain why a method performs well or poorly for this data set.

Getting Started

The code below is starter code. It is intended to help you begin the analysis, not to fully solve the lab. You should modify it, add to it, and justify the choices you make.

For each problem, consider giving:

1. The code needed to produce your result.
2. At least one figure or table when appropriate.
3. A short interpretation in words.
4. A clear statement of the preprocessing or tuning choices you used.

Problems

Problem 0: Explore and preprocess the data

- Problem 0a - What are the dimensions of the author word-count matrix? How many texts and how many words are included?
- Problem 0b - Are the word counts highly skewed? Use histograms or summary statistics to support your answer.
- Problem 0c - Compare raw counts and log-transformed counts. Which representation seems more appropriate for clustering? Why?
- Problem 0d - Should the word columns be centered or scaled before applying each method? Explain why the answer may differ across PCA, MDS, K-means, hierarchical clustering, heatmaps, and NMF.

Problem 1: Visualization

- Problem 1a - We wish to plot the author texts and the words in two dimensions. Which methods are appropriate for visualizing texts? Which methods are appropriate for visualizing words? Why?

- Problem 1b - Apply PCA to visualize the author texts. Compare at least two preprocessing choices, such as raw counts versus log-transformed counts or unscaled versus scaled variables. Explain the results.
- Problem 1c - Apply classical MDS to visualize the author texts. Try more than one distance metric and interpret the results.
- Problem 1d - Can MDS help you decide which distance is appropriate for this data? Which distance do you prefer and why?
- Problem 1e - Apply MDS with your chosen distance to visualize the words. Which words appear unusual or important? How should the word plot be interpreted?

Problem 2: K-means

- Problem 2a - Apply K-means with K = 4 to this data.
- Problem 2b - How well does K-means separate the authors? Use a table and a visualization.
- Problem 2c - Does K-means give stable results across random starts? Demonstrate this with repeated runs or by using `nstart`.
- Problem 2d - Is K-means an appropriate clustering algorithm for this data? Why or why not?
- Problem 2e - Optional: Use an elbow plot or silhouette plot to examine whether K = 4 is supported by the data.

Problem 3: Hierarchical clustering

- Problem 3a - Apply hierarchical clustering to this data set.
- Problem 3b - Which distance is best to use? Why?
- Problem 3c - Which linkage is best to use? Why?
- Problem 3d - Do any linkages perform particularly poorly? Explain why this might happen.
- Problem 3e - Visualize your hierarchical clustering results using a dendrogram and a low-dimensional plot.

Problem 4: Cluster heatmaps and biclustering-style visualization

- Problem 4a - Apply a clustered heatmap to visualize the data. Which distance and linkage functions did you use?
- Problem 4b - Should you use all words or a subset of informative words? Explain your choice.
- Problem 4c - Interpret the cluster heatmap. Which words appear important for distinguishing author texts?
- Problem 4d - How does the heatmap complement the PCA, MDS, K-means, and hierarchical clustering results?

Problem 5: Nonnegative matrix factorization

- Problem 5a - Apply NMF with K = 4 and use the W matrix to assign cluster labels to each observation.
- Problem 5b - How well does NMF perform? Interpret and explain this result.
- Problem 5c - Use the H matrix to identify words that contribute strongly to each factor. Which words appear important for distinguishing authors?
- Problem 5d - Optional: Try K = 3 and K = 5. Does your interpretation change?

Problem 6: Wrap-up

- Problem 6a - Overall, which method is best at clustering the author texts? Why?
- Problem 6b - Which preprocessing choices were most important?
- Problem 6c - Which words are key for distinguishing the author texts? How did you determine these?
- Problem 6d - Which method gives the best visual summary of the data?
- Problem 6e - What would you do differently if the true author labels were not available at all?

Starter R code for the clustering lab

Do not peek if you want to practice coding on your own!

The following code provides a scaffold. It deliberately leaves some decisions for you, including which input matrix to cluster, which distance metric to use, and which linkage method to prefer.

Load packages

```
pkg_available <- function(pkg) requireNamespace(pkg, quietly = TRUE)

required_pkgs <- c("ggplot2")
optional_pkgs <- c("NMF", "cluster", "mclust", "ggrepel", "pheatmap", "umap")

missing_required <- required_pkgs[!vapply(required_pkgs, pkg_available, logical(1))]
if (length(missing_required) > 0) {
  stop("Please install required package(s): ", paste(missing_required, collapse = ", "))
}

invisible(lapply(required_pkgs, library, character.only = TRUE))

missing_optional <- optional_pkgs[!vapply(optional_pkgs, pkg_available, logical(1))]
if (length(missing_optional) > 0) {
  message("Optional package(s) not installed: ", paste(missing_optional, collapse = ", "))
  message("Some optional starter chunks will be skipped.")
}

# The NMF package expects to be attached for some internal setup steps.
# Calling functions only as NMF::nmf() can fail on some systems during knitting.
if (pkg_available("NMF")) {
  suppressPackageStartupMessages(library(NMF))
}
```

Load and inspect the data

```
load("UnsupL_SISBID_2026.Rdata")
stopifnot(exists("author"))

# Basic inspection
class(author)
```

```
## [1] "matrix" "array"
```

```
dim(author)
```

```
## [1] 841  70
```

```
head(author[, 1:min(6, ncol(author))])
```

```
##         a all also an and any
## Austen 46  12    0  3  66   9
## Austen 35  10    0  7  44   4
## Austen 46   2    0  3  40   1
## Austen 40   7    0  4  64   3
## Austen 29   5    0  6  52   5
## Austen 27   8    0  3  42   2
```

```r
colnames(author)
```

```
##  [1] "a"      "all"    "also"   "an"     "and"    "any"    "are"    "as"
##  [9] "at"     "be"     "been"   "but"    "by"     "can"    "do"     "down"
## [17] "even"   "every"  "for."   "from"   "had"    "has"    "have"   "her"
## [25] "his"    "if."    "in."    "into"   "is"     "it"     "its"    "may"
## [33] "more"   "must"   "my"     "no"     "not"    "now"    "of"     "on"
## [41] "one"    "only"   "or"     "our"    "should" "so"     "some"   "such"
## [49] "than"   "that"   "the"    "their"  "then"   "there"  "things" "this"
## [57] "to"     "up"     "upon"   "was"    "were"   "what"   "when"   "which"
## [65] "who"    "will"   "with"   "would"  "your"   "BookID"
```

```r
unique(rownames(author))
```

```
## [1] "Austen"      "London"      "Milton"      "Shakespeare"
```

```r
# True labels are available for evaluation, but do not use them to build clusters.
# The row names of this data set are author labels, so they are not unique.
# Keep the labels for evaluation, but create unique document IDs for plotting
# and heatmap annotations.
TrueAuth <- factor(rownames(author))
DocID <- make.unique(as.character(TrueAuth), sep = "_")
table(TrueAuth)
```

```
## TrueAuth
##      Austen      London      Milton Shakespeare
##         317         296          55         173
```

Create analysis matrices

```r
# Remove non-word identifier columns if present.
# Use exact matches so genuine word columns such as "did" are not removed.
id_names <- c("bookid", "book_id", "chapterid", "chapter_id",
              "textid", "text_id", "docid", "doc_id", "id")
id_cols <- which(tolower(colnames(author)) %in% id_names)
word_cols <- setdiff(seq_len(ncol(author)), id_cols)

AuthorData <- as.matrix(author[, word_cols, drop = FALSE])
storage.mode(AuthorData) <- "numeric"
rownames(AuthorData) <- DocID

# Several useful representations. Use the same unique document IDs for each
# representation so downstream plots, dendrograms, and heatmaps can align rows.
X_raw <- AuthorData
X_log <- log1p(AuthorData)
X_log_scaled <- scale(X_log)
rownames(X_log_scaled) <- DocID

# Choose one default representation to start with.
# You should change this and compare results.
X_work <- X_log

list(
  raw_dim = dim(X_raw),
  log_dim = dim(X_log),
```

```
  scaled_dim = dim(X_log_scaled)
)
```

```
## $raw_dim
## [1] 841  69
##
## $log_dim
## [1] 841  69
##
## $scaled_dim
## [1] 841  69
```

Problem 0 starter: explore word counts

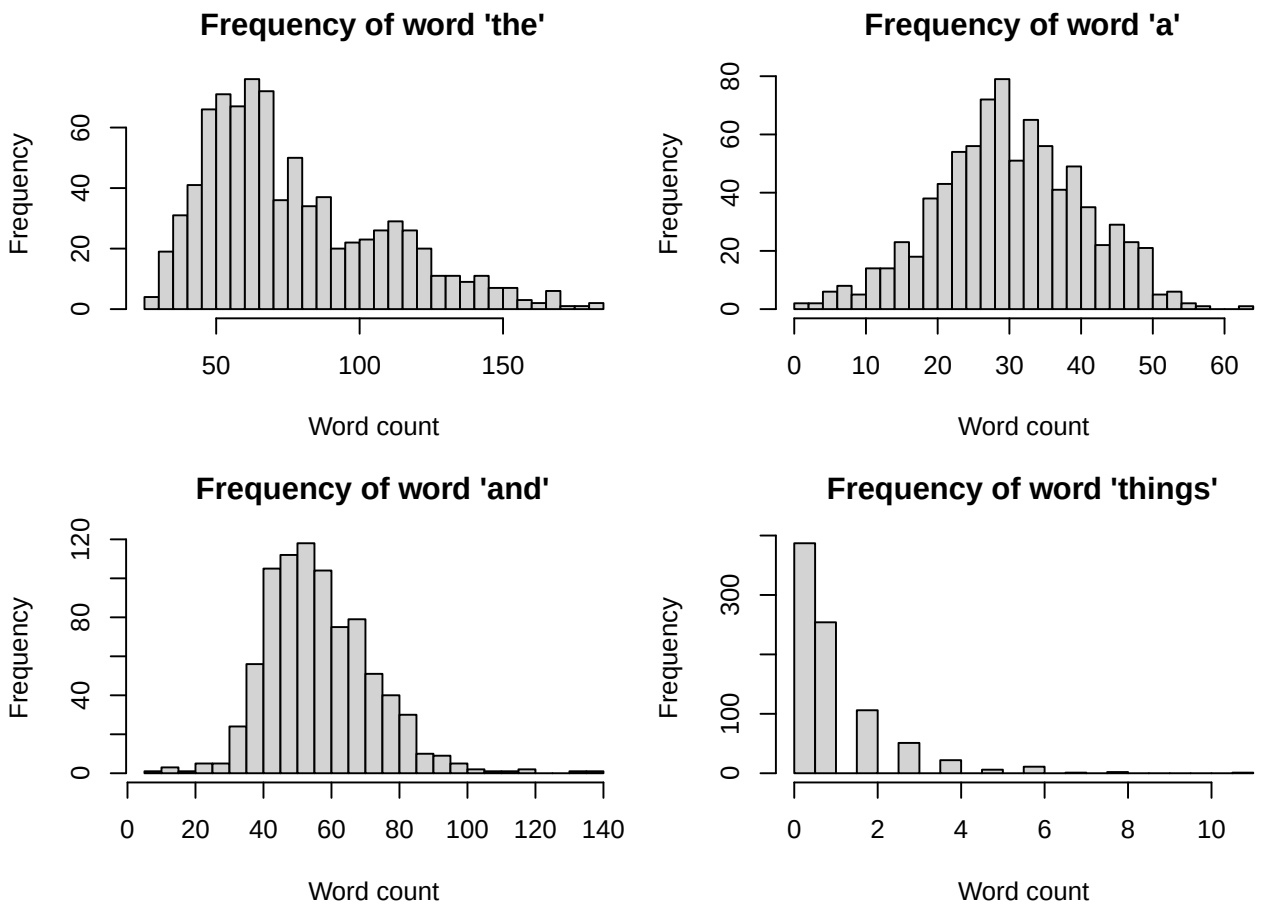
```
summary(as.vector(AuthorData))
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##    0.00    2.00    5.00   10.21   12.00  183.00
```

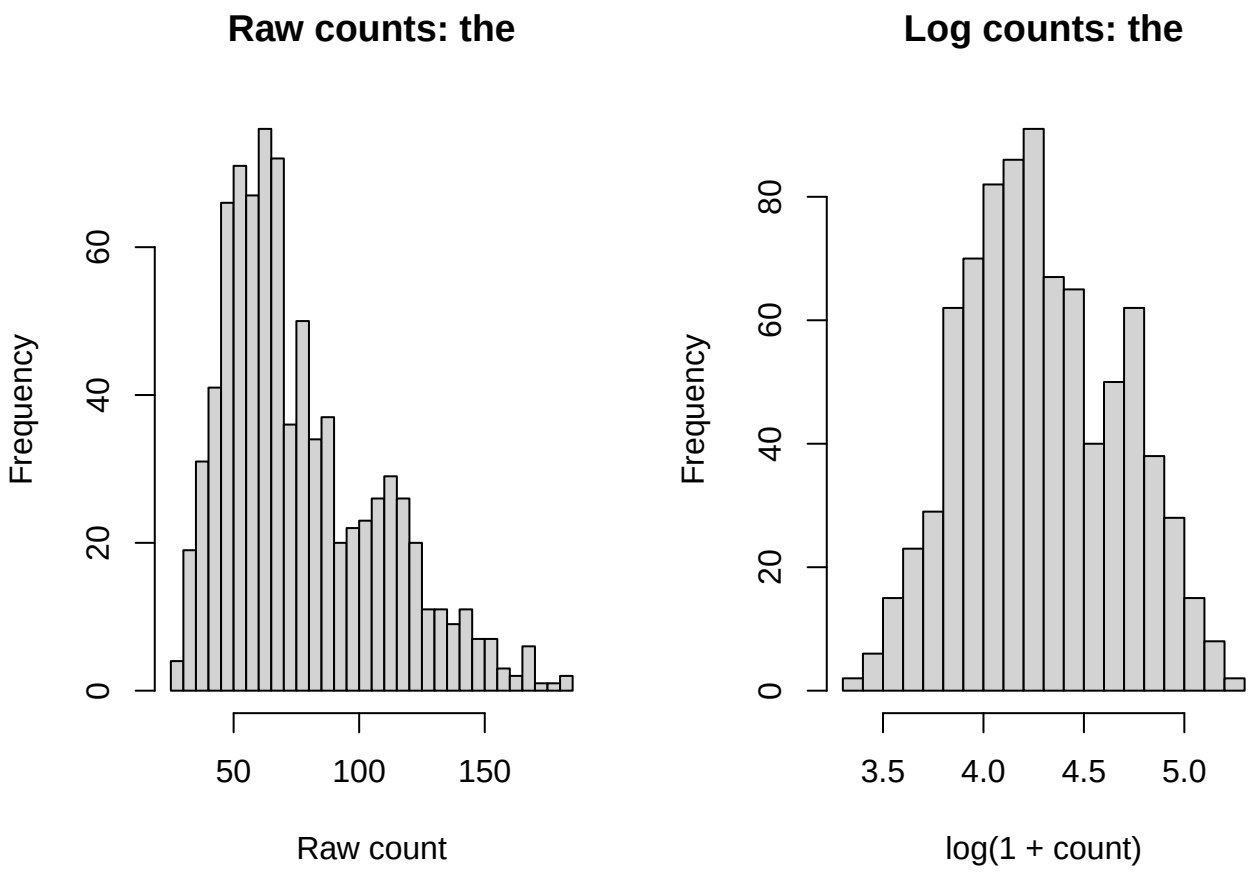
```
summary(as.vector(X_log))
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   0.000   1.099   1.792   1.817   2.565   5.215
```

```
plot_word_histograms <- function(words, dat = AuthorData, breaks = 25) {
  words <- intersect(words, colnames(dat))
  if (length(words) == 0) {
    stop("None of the requested words were found in the data.")
  }
  old_par <- par(mfrow = c(ceiling(length(words) / 2), 2), mar = c(4, 4, 3, 1))
  on.exit(par(old_par))
  for (w in words) {
    hist(dat[, w], breaks = breaks,
         main = paste("Frequency of word", shQuote(w)),
         xlab = "Word count")
  }
}

plot_word_histograms(c("the", "a", "and", "things"))
```




```
# Compare raw and log-transformed distributions for one word.
word_to_plot <- "the"
if (word_to_plot %in% colnames(AuthorData)) {
  par(mfrow = c(1, 2))
  hist(AuthorData[, word_to_plot], breaks = 25,
       main = paste("Raw counts:", word_to_plot), xlab = "Raw count")
  hist(log1p(AuthorData[, word_to_plot]), breaks = 25,
       main = paste("Log counts:", word_to_plot), xlab = "log(1 + count)")
  par(mfrow = c(1, 1))
}
```



Helper functions for plots and evaluation

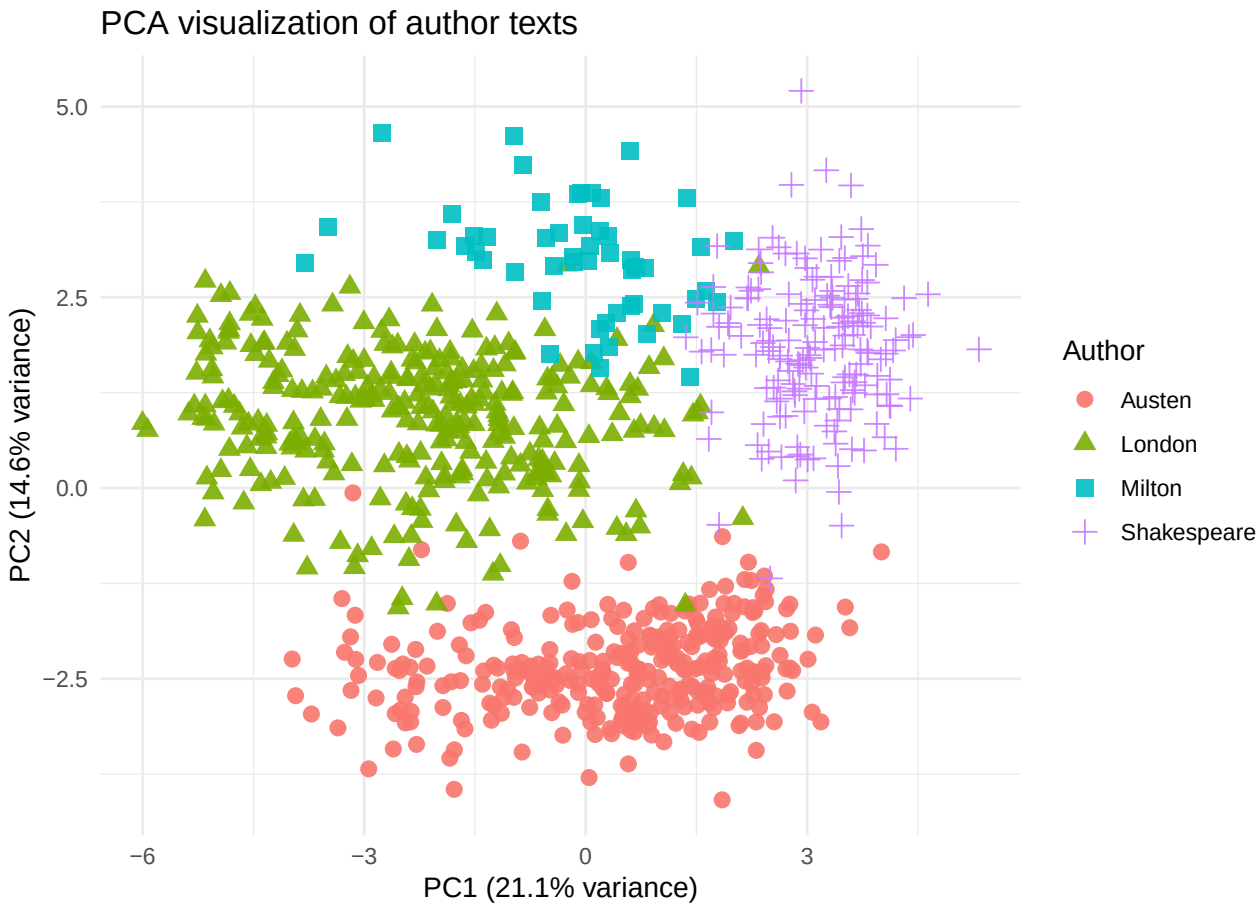
```
make_cluster_plot <- function(coords, cluster, truth = TrueAuth, title = "Cluster plot") {
  plot_dat <- data.frame(
    Dim1 = coords[, 1],
    Dim2 = coords[, 2],
    Cluster = factor(cluster),
    TrueAuthor = truth
  )
  ggplot(plot_dat, aes(x = Dim1, y = Dim2, color = Cluster, shape = TrueAuthor)) +
    geom_point(size = 2.8, alpha = 0.9) +
    labs(title = title, x = "Dimension 1", y = "Dimension 2") +
    theme_minimal()
}

make_truth_plot <- function(coords, truth = TrueAuth, title = "Low-dimensional view") {
  plot_dat <- data.frame(
    Dim1 = coords[, 1],
    Dim2 = coords[, 2],
    TrueAuthor = truth
  )
  ggplot(plot_dat, aes(x = Dim1, y = Dim2, color = TrueAuthor, shape = TrueAuthor)) +
    geom_point(size = 2.8, alpha = 0.9) +
    labs(title = title, x = "Dimension 1", y = "Dimension 2") +
    theme_minimal()
}
```

```r
cluster_summary <- function(cluster, truth = TrueAuth) {
  tab <- table(cluster, truth)
  print(tab)
  if (pkg_available("mclust")) {
    ari <- mclust::adjustedRandIndex(cluster, truth)
    cat("Adjusted Rand index:", round(ari, 3), "\n")
  } else {
    cat("Install the mclust package for adjusted Rand index.\n")
  }
  invisible(tab)
}
```

Problem 1 starter: visualization

PCA for author texts

```r
# Start with log-transformed counts. Try changing X_work to X_raw or X_log_scaled.
pca_fit <- prcomp(X_work, center = TRUE, scale. = FALSE)

pve <- pca_fit$sdev^2 / sum(pca_fit$sdev^2)
pca_scores <- pca_fit$x[, 1:2]

pca_dat <- data.frame(
  PC1 = pca_scores[, 1],
  PC2 = pca_scores[, 2],
  Author = TrueAuth
)

ggplot(pca_dat, aes(x = PC1, y = PC2, color = Author, shape = Author)) +
  geom_point(size = 2.8, alpha = 0.9) +
  labs(
    title = "PCA visualization of author texts",
    x = paste0("PC1 (", round(100 * pve[1], 1), "% variance)"),
    y = paste0("PC2 (", round(100 * pve[2], 1), "% variance)")
  ) +
  theme_minimal()
```

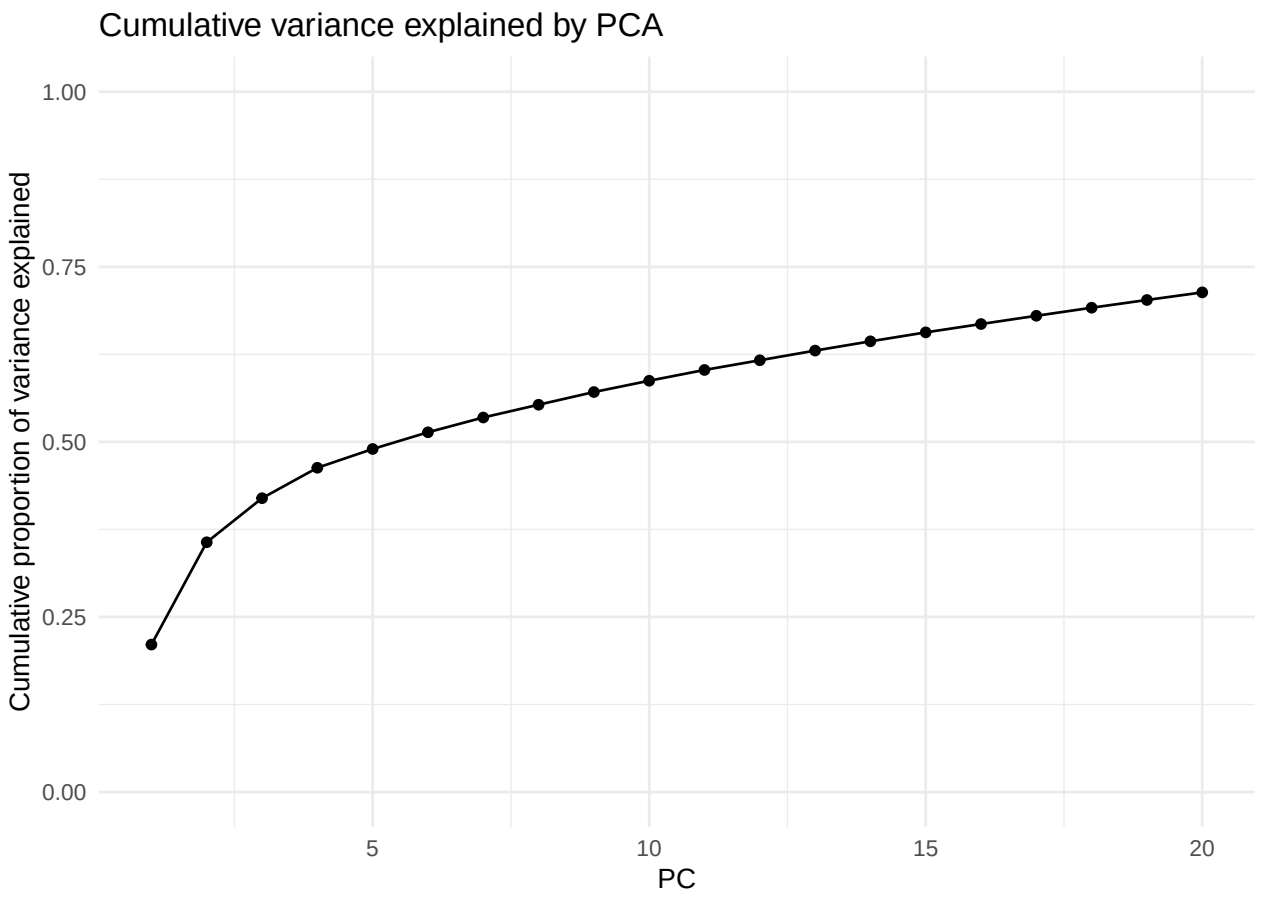



```
var_dat <- data.frame(
  PC = seq_along(pve),
  PVE = pve,
  Cumulative = cumsum(pve)
)

head(var_dat, 10)
```

```
##    PC        PVE Cumulative
## 1   1 0.21057356  0.2105736
## 2   2 0.14611211  0.3566857
## 3   3 0.06289925  0.4195849
## 4   4 0.04344345  0.4630284
## 5   5 0.02680850  0.4898369
## 6   6 0.02386685  0.5137037
## 7   7 0.02110162  0.5348053
## 8   8 0.01825681  0.5530621
## 9   9 0.01808301  0.5711452
## 10 10 0.01614706  0.5872922
```

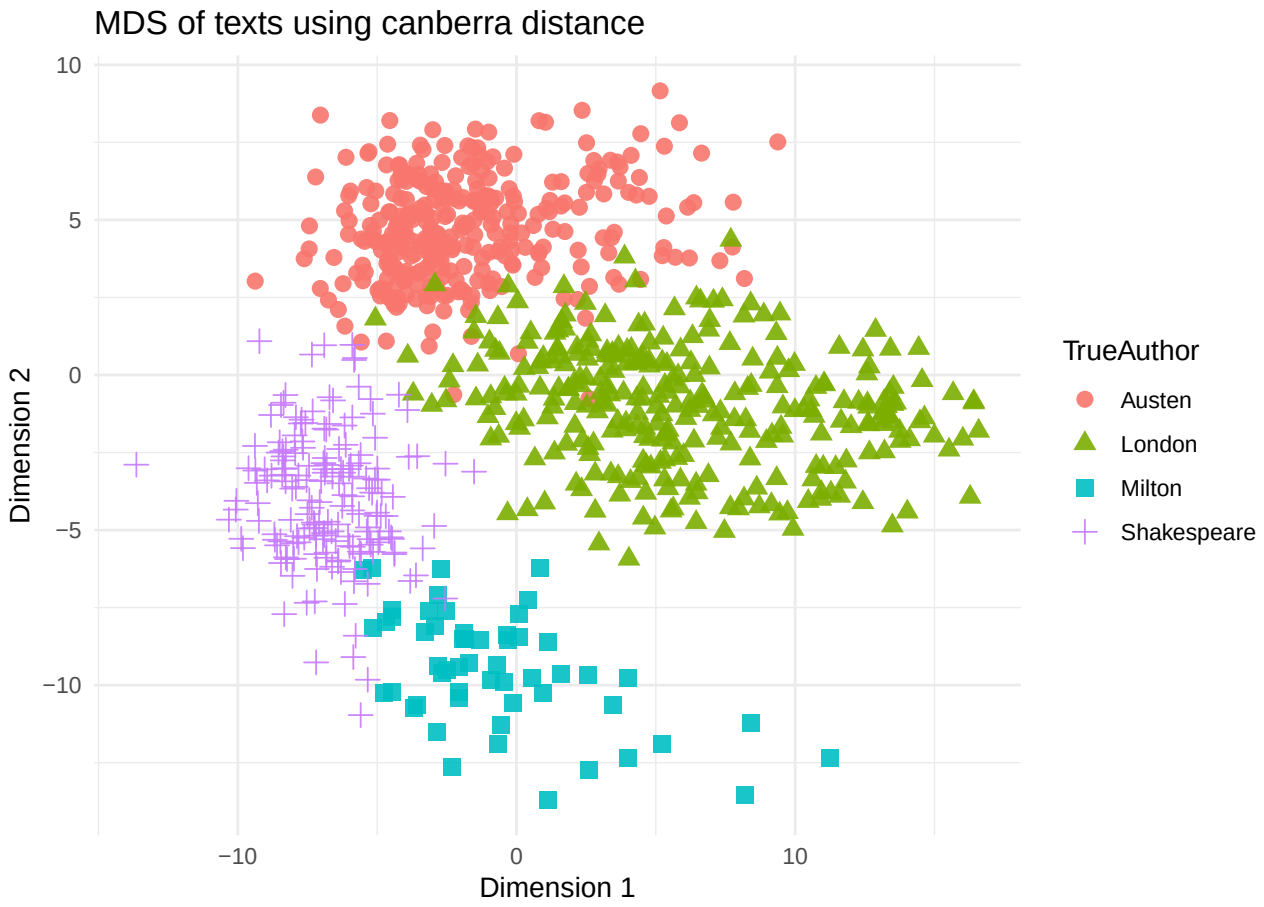
```
ggplot(var_dat[1:min(20, nrow(var_dat)), ], aes(x = PC, y = Cumulative)) +
  geom_point() +
  geom_line() +
  ylim(0, 1) +
  labs(title = "Cumulative variance explained by PCA",
       y = "Cumulative proportion of variance explained") +
```

```
  theme_minimal()
```

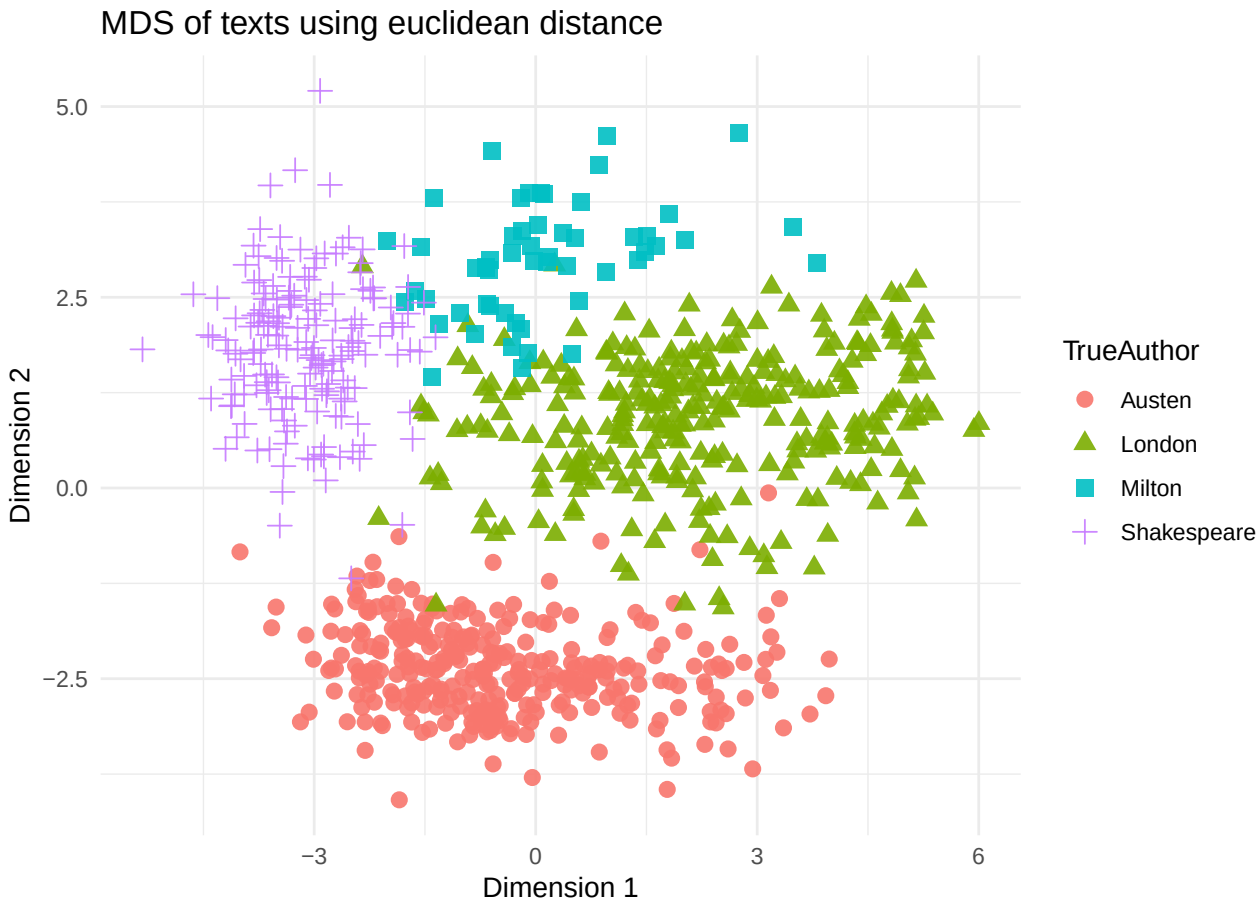


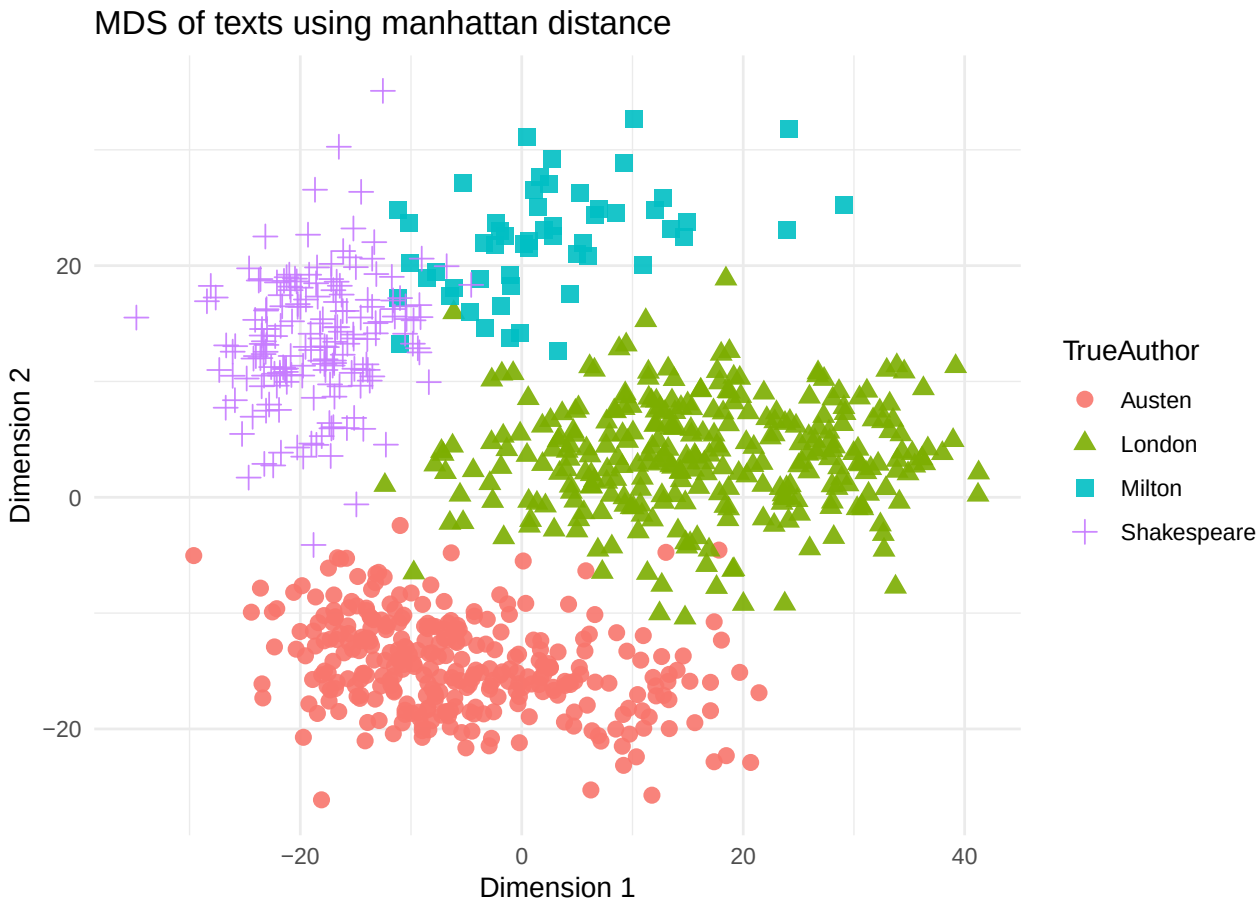
Classical MDS for author texts

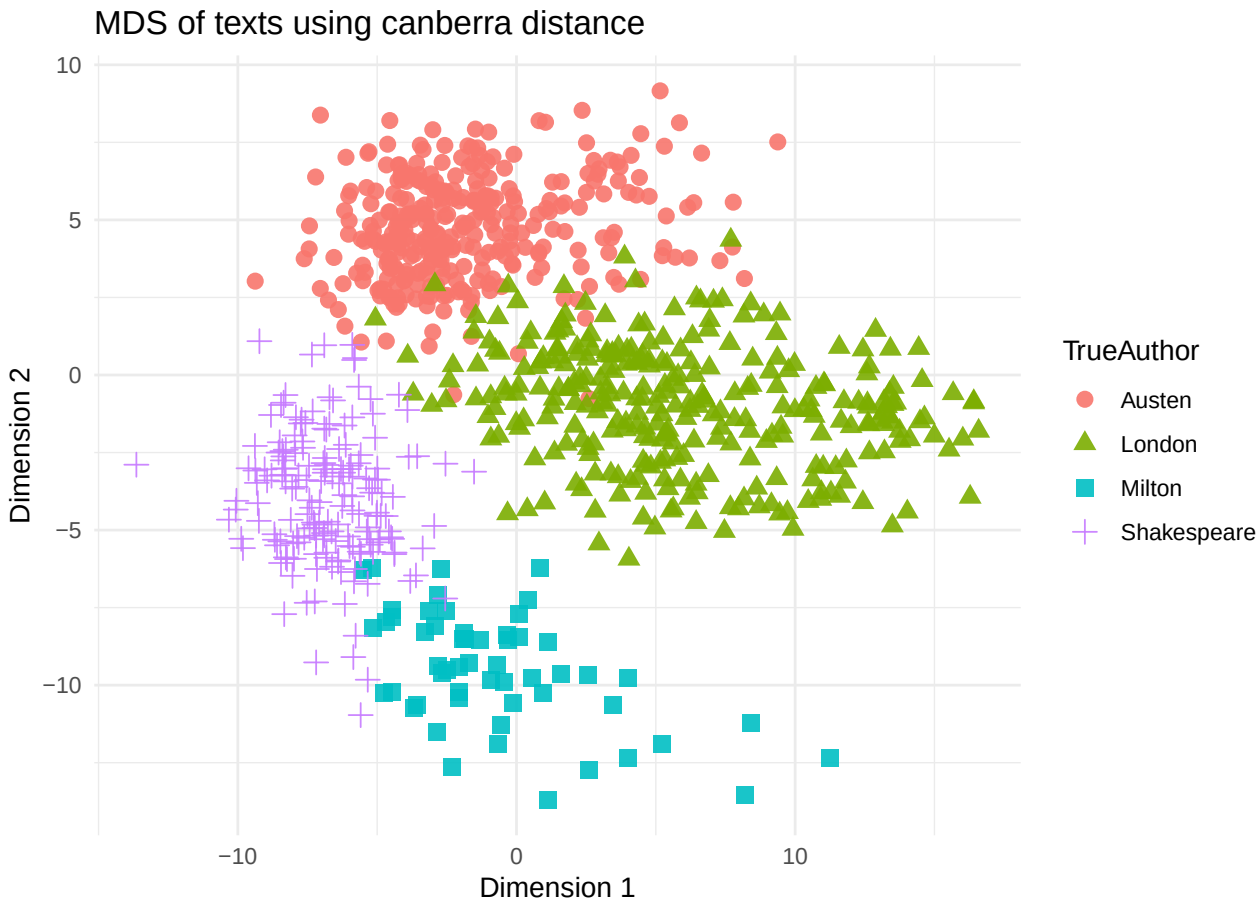
```
run_mds <- function(dat, distance = "canberra", k = 2) {
  dmat <- dist(dat, method = distance)
  fit <- cmdscale(dmat, k = k, eig = TRUE)
  coords <- fit$points
  colnames(coords) <- paste0("MDS", seq_len(ncol(coords)))
  list(distance = distance, dmat = dmat, fit = fit, coords = coords)
}

# Try several distances and decide which you prefer.
distance_choices <- c("euclidean", "manhattan", "canberra")
chosen_distance <- "canberra"

mds_res <- run_mds(X_log, distance = chosen_distance)
make_truth_plot(mds_res$coords, title = paste("MDS of texts using", chosen_distance, "distance"))
```




```
# This code gives a visual comparison, but you still need to interpret the plots.
mds_list <- lapply(distance_choices, function(d) run_mds(X_log, distance = d))

for (i in seq_along(mds_list)) {
  print(make_truth_plot(
    mds_list[[i]]$coords,
    title = paste("MDS of texts using", mds_list[[i]]$distance, "distance")
  ))
}
```


MDS of texts using euclidean distance
5.0
2.5
0.0
−2.5
Dimension 2
−3
0
3
6
Dimension 1
TrueAuthor
Austen
London
Milton
Shakespeare


MDS of texts using manhattan distance
Dimension 2
20
0
−20
−20
0
20
40
Dimension 1
TrueAuthor
Austen
London
Milton
Shakespeare

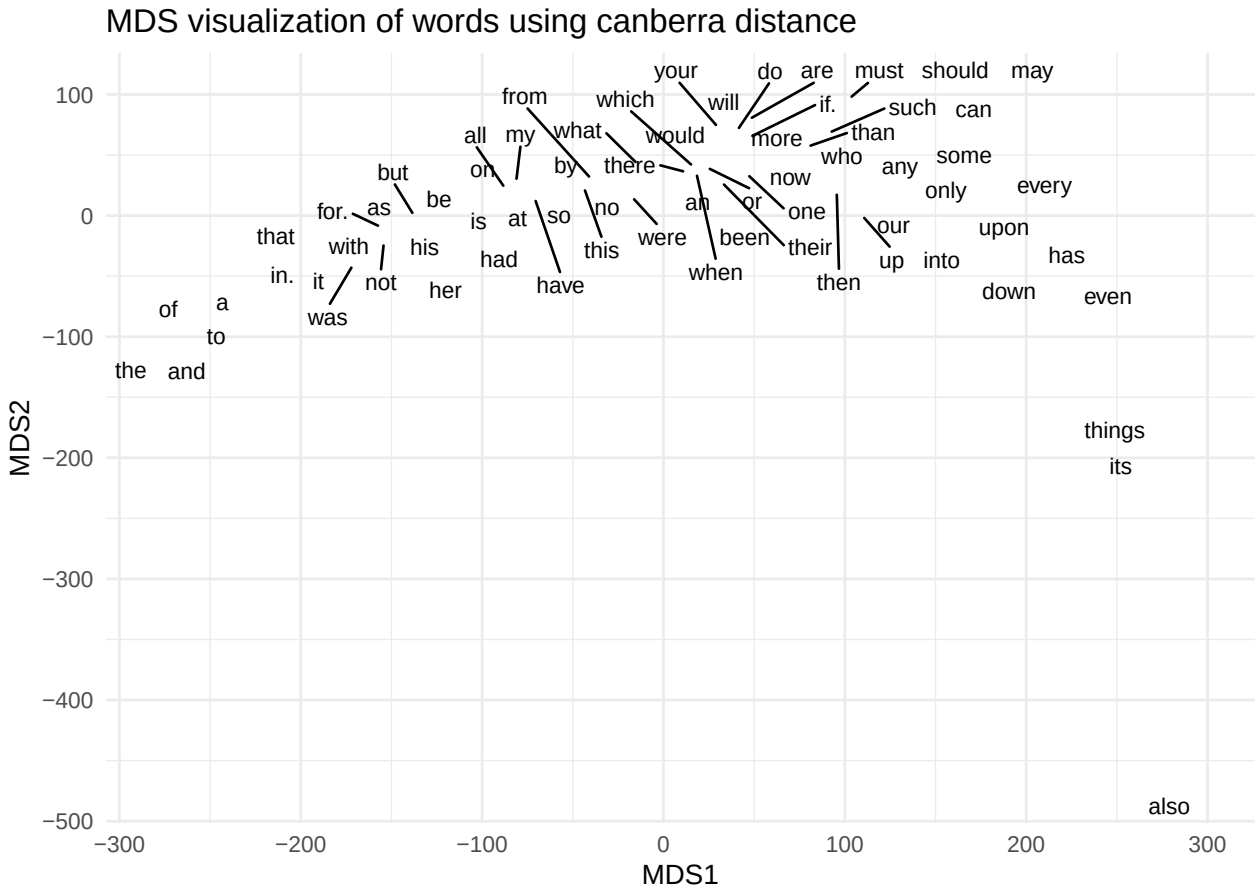


Optional UMAP visualization

```
if (pkg_available("umap")) {
  set.seed(2026)
  umap_res <- umap::umap(X_log)
  make_truth_plot(umap_res$layout, title = "UMAP visualization of author texts")
}
```

MDS for words

```
# Rows should be words and columns should be texts when visualizing words.
# Try raw counts, log counts, or scaled log counts and compare.
WordData <- t(X_log)
word_mds <- run_mds(WordData, distance = chosen_distance)

word_plot_dat <- data.frame(
  MDS1 = word_mds$coords[, 1],
  MDS2 = word_mds$coords[, 2],
  Word = rownames(word_mds$coords)
)

p_words <- ggplot(word_plot_dat, aes(x = MDS1, y = MDS2, label = Word)) +
  labs(title = paste("MDS visualization of words using", chosen_distance, "distance")) +
  theme_minimal()

if (pkg_available("ggrepel")) {
  p_words + ggrepel::geom_text_repel(max.overlaps = 40, size = 3)
} else {
  p_words + geom_text(size = 3, check_overlap = TRUE)
}
```

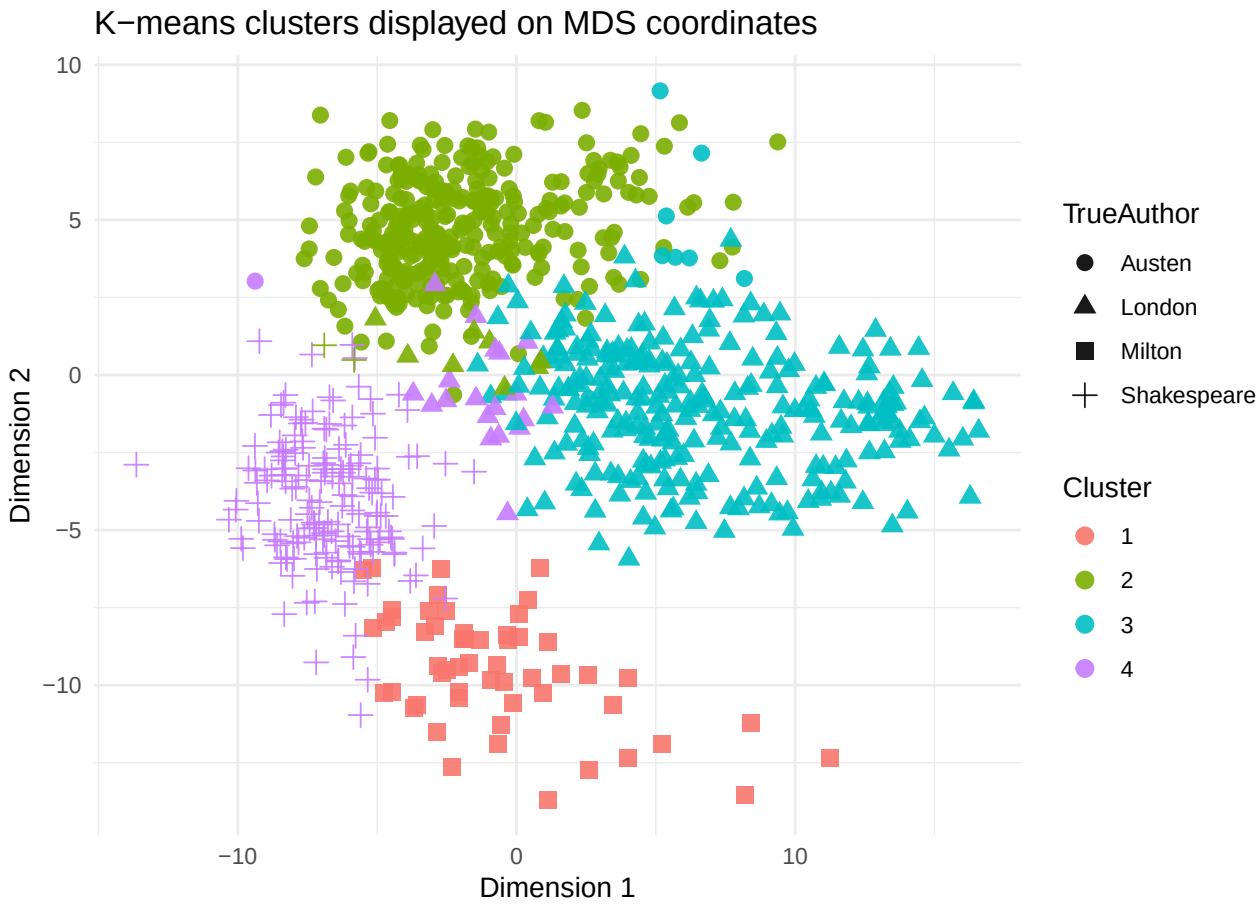


Problem 2 starter: K-means

```
K <- 4
set.seed(2026)

# Important: try X_raw, X_log, and X_log_scaled and compare.
km_input <- X_log

km <- kmeans(km_input, centers = K, nstart = 5)
cluster_summary(km$cluster)
```

```
##         truth
## cluster Austen London Milton Shakespeare
##       1      0      0     55           0
##       2    308      8      0           2
##       3      8    269      0           0
##       4      1     19      0         171
## Adjusted Rand index: 0.873
```

```
make_cluster_plot(
  mds_res$coords,
  cluster = km$cluster,
  title = "K-means clusters displayed on MDS coordinates"
)
```

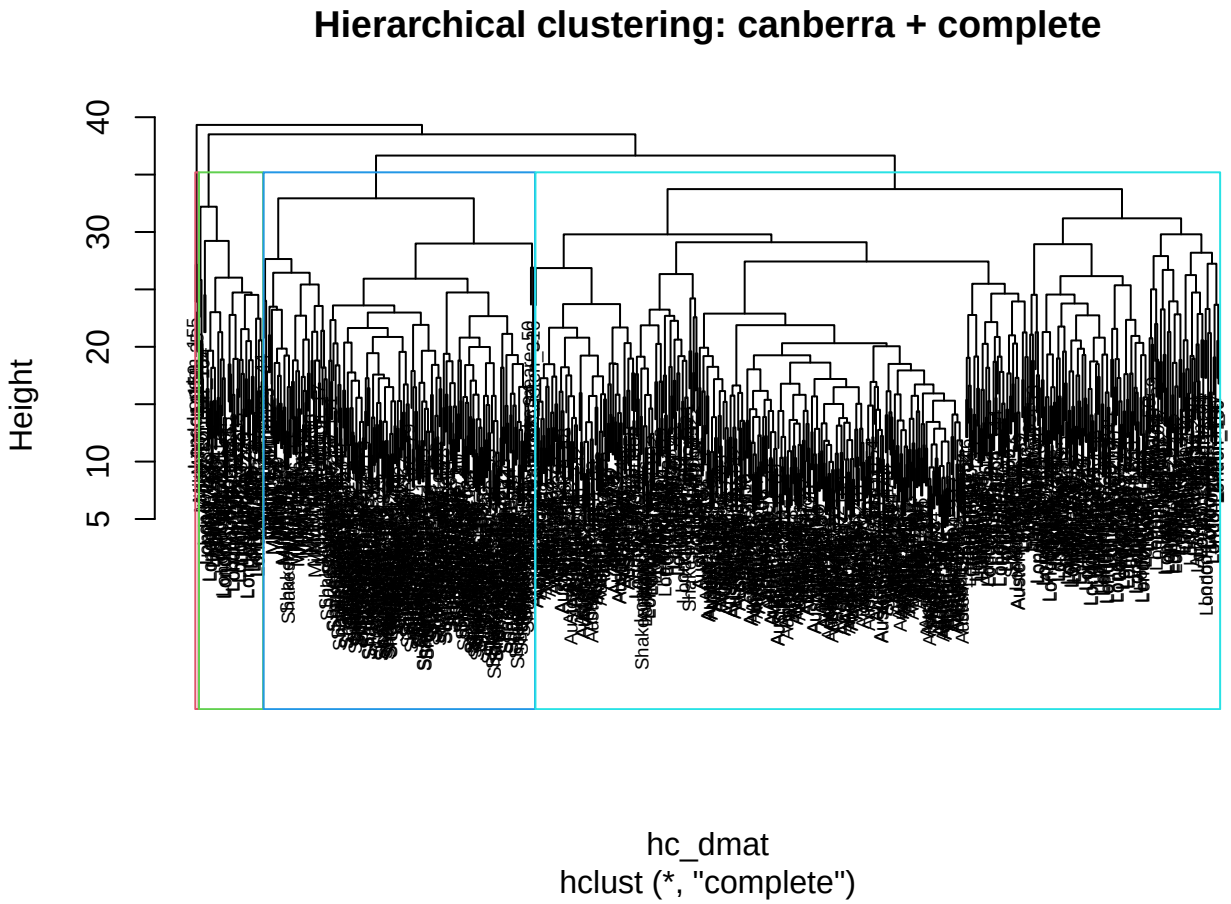


Problem 3 starter: hierarchical clustering

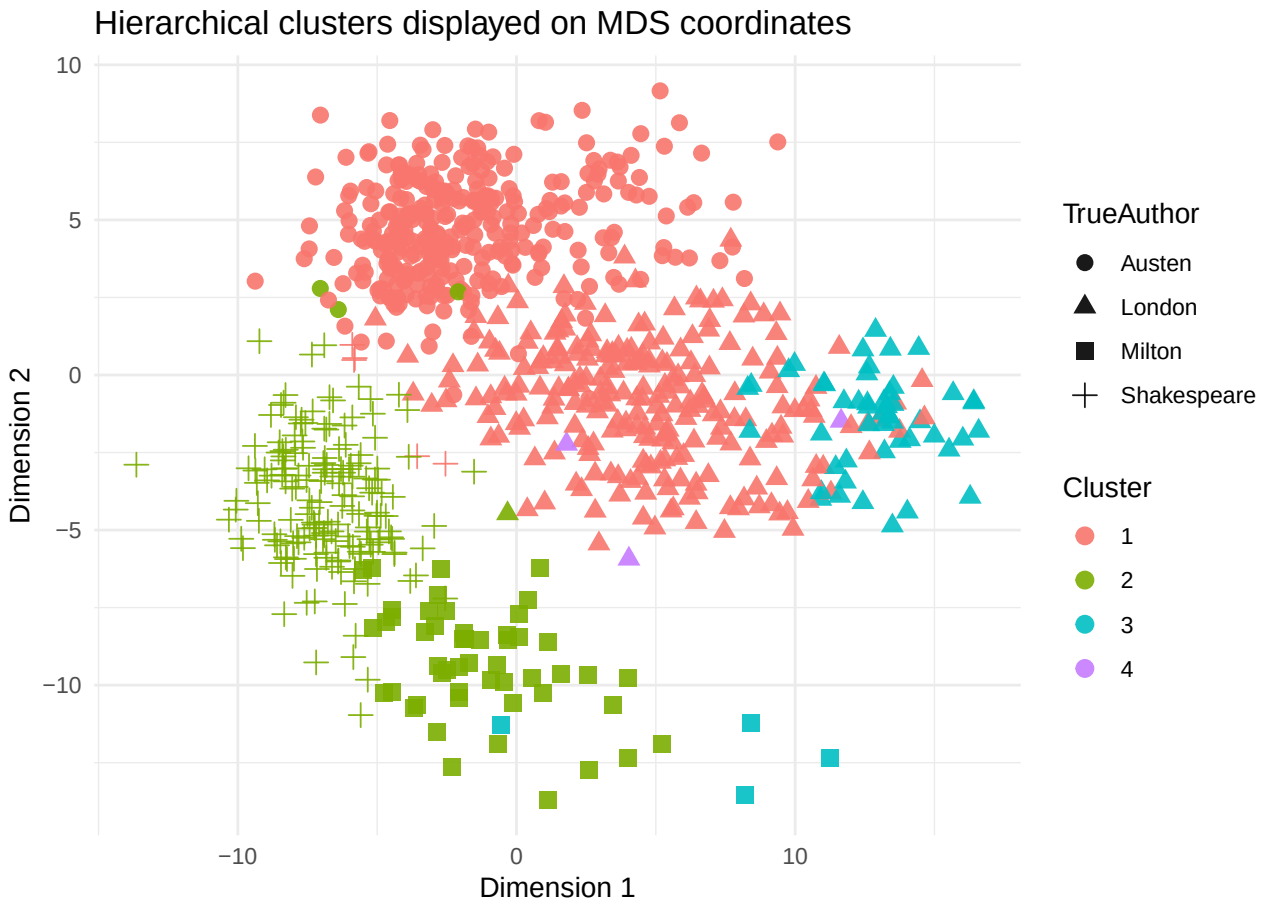
```
# Try changing these values.
hc_distance <- chosen_distance
hc_linkage <- "complete"  # try "single", "average", "complete", "ward.D2"

hc_dmat <- dist(X_log, method = hc_distance)
hc_fit <- hclust(hc_dmat, method = hc_linkage)
hc_cluster <- cutree(hc_fit, k = 4)

cluster_summary(hc_cluster)
```

```
##         truth
## cluster Austen London Milton Shakespeare
##       1    314    243      0           5
##       2      3      1     51         168
##       3      0     49      4           0
##       4      0      3      0           0
## Adjusted Rand index: 0.421
```

```
plot(hc_fit, cex = 0.6, main = paste("Hierarchical clustering:", hc_distance, "+", hc_linkage))
rect.hclust(hc_fit, k = 4, border = 2:5)
```




```
make_cluster_plot(
  mds_res$coords,
  cluster = hc_cluster,
  title = paste("Hierarchical clusters displayed on MDS coordinates")
)
```

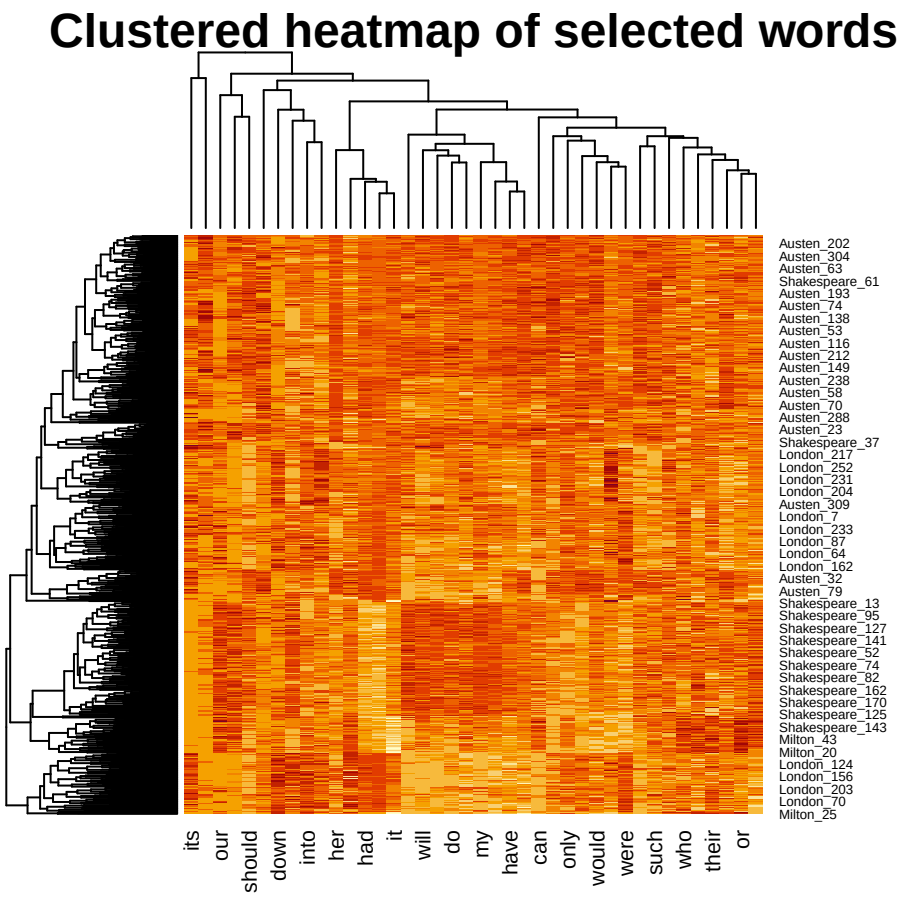



```
# Optional comparison scaffold.
# Run this after you understand the basic hierarchical clustering code above.
# Then decide which distance and linkage combination you prefer.

distance_grid <- c("euclidean", "manhattan", "canberra")
linkage_grid <- c("single", "average", "complete", "ward.D2")

hc_grid <- expand.grid(
  distance = distance_grid,
  linkage = linkage_grid,
  stringsAsFactors = FALSE
)

hc_results <- lapply(seq_len(nrow(hc_grid)), function(i) {
  d <- hc_grid$distance[i]
  l <- hc_grid$linkage[i]
  fit <- hclust(dist(X_log, method = d), method = l)
  cl <- cutree(fit, k = 4)
  data.frame(
    distance = d,
    linkage = l,
    ari = if (pkg_available("mclust")) mclust::adjustedRandIndex(cl, TrueAuth) else NA_real_
  )
})
```

```r
do.call(rbind, hc_results)
```

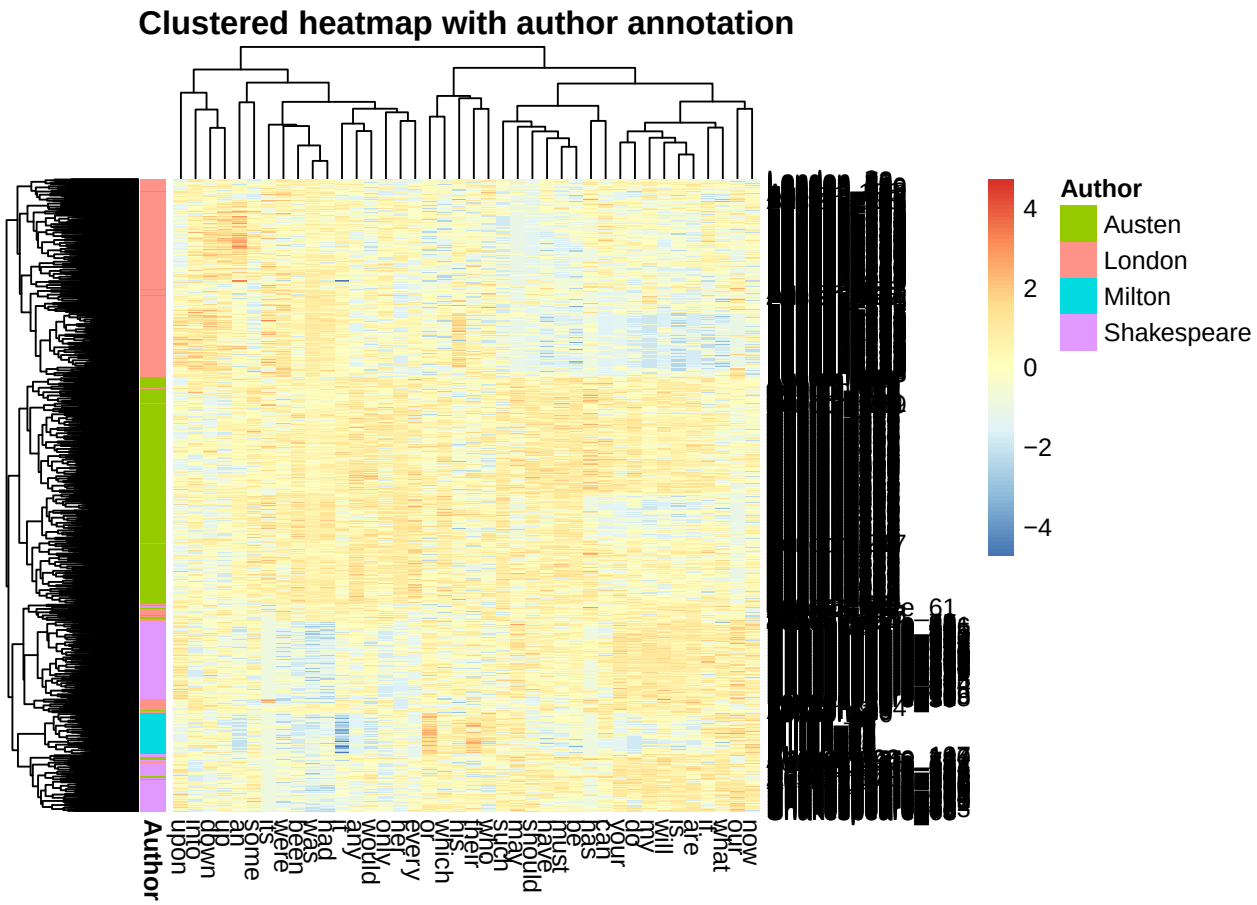
Problem 4 starter: clustered heatmaps

```r
# Heatmaps are easier to read if we use a subset of words.
# Here we start with the most variable words, but you should justify your choice.
top_word_count <- min(40, ncol(X_log))
word_variance <- apply(X_log, 2, var)
top_words <- names(sort(word_variance, decreasing = TRUE))[1:top_word_count]

heatmap_mat <- X_log[, top_words, drop = FALSE]

heatmap(
  heatmap_mat,
  distfun = function(x) dist(x, method = chosen_distance),
  hclustfun = function(x) hclust(x, method = "complete"),
  scale = "column",
  main = "Clustered heatmap of selected words"
)
```




```r
if (pkg_available("pheatmap")) {
  # pheatmap annotations are matched by row name. Because author labels repeat,
  # use the unique document IDs as annotation row names and store author labels
  # in a separate column.
```

```
  ann_row <- data.frame(
    Author = TrueAuth,
    row.names = rownames(heatmap_mat)
  )

  pheatmap::pheatmap(
    heatmap_mat,
    scale = "column",
    clustering_distance_rows = chosen_distance,
    clustering_distance_cols = "correlation",
    clustering_method = "complete",
    annotation_row = ann_row,
    main = "Clustered heatmap with author annotation"
  )
}
```




```
# Starter list of high-variance words.
# Do not stop here: compare this list with loadings, NMF factors, and heatmap patterns.
head(sort(word_variance, decreasing = TRUE), 20)
```

```
##       her        my      your       was       our        is      will       had
## 1.7075649 1.1487453 0.9769817 0.9637879 0.8374804 0.8273374 0.7760981 0.7759710
##       are        an        do      been      were      have     their       any
## 0.6551451 0.6210663 0.6023162 0.5998821 0.5470507 0.5317935 0.5237313 0.5170393
##      must     would    should       may
## 0.5139985 0.5117279 0.5097792 0.5084065
```

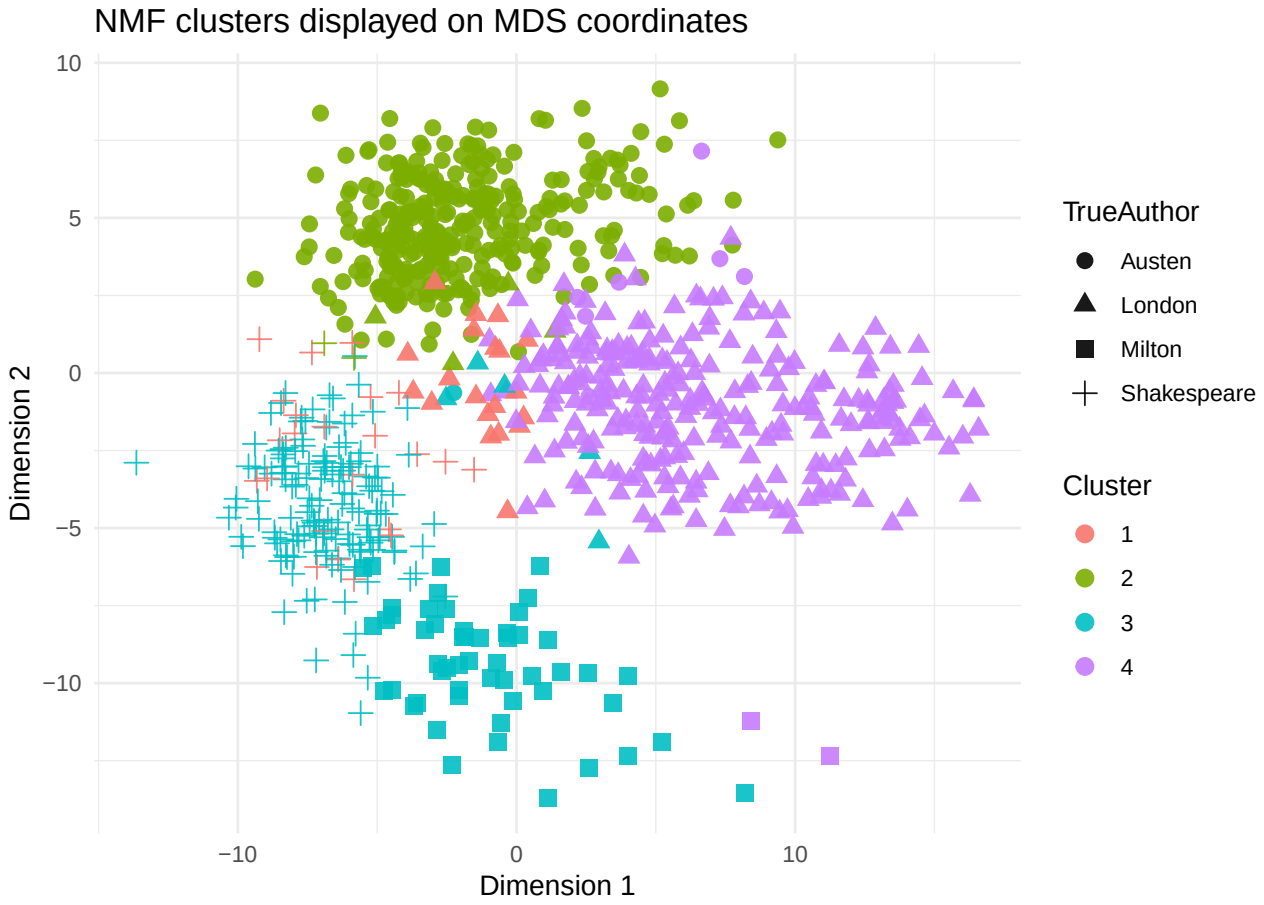
Problem 5 starter: NMF

NMF requires nonnegative input, so do not center or scale the data before running NMF. Log-transformed counts are often a useful starting point because they remain nonnegative.

```
if (pkg_available("NMF")) {
  set.seed(2026)
  nmf_input <- X_log
  nmf_fit <- nmf(nmf_input, rank = 4, nrun = 10, seed = 2026)

  W <- basis(nmf_fit)
  H <- coef(nmf_fit)

  nmf_cluster <- apply(W, 1, which.max)
  cluster_summary(nmf_cluster)
}
```

```
##         truth
## cluster Austen London Milton Shakespeare
##       1      0     20      0          23
##       2    309      4      0           2
##       3      1      5     53         148
##       4      7    267      2           0
## Adjusted Rand index: 0.815
```

```
if (pkg_available("NMF")) {
  make_cluster_plot(
    mds_res$coords,
    cluster = nmf_cluster,
    title = "NMF clusters displayed on MDS coordinates"
  )
}
```

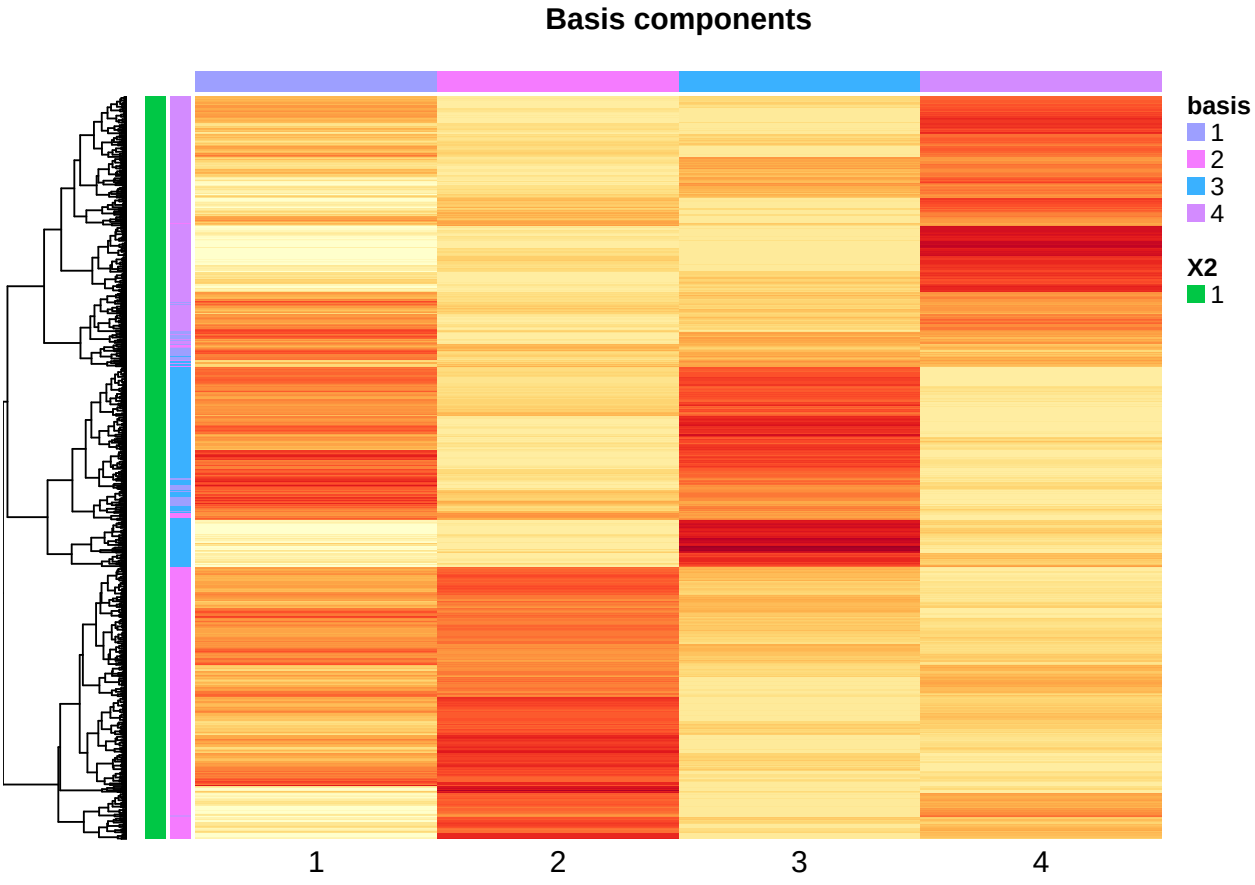


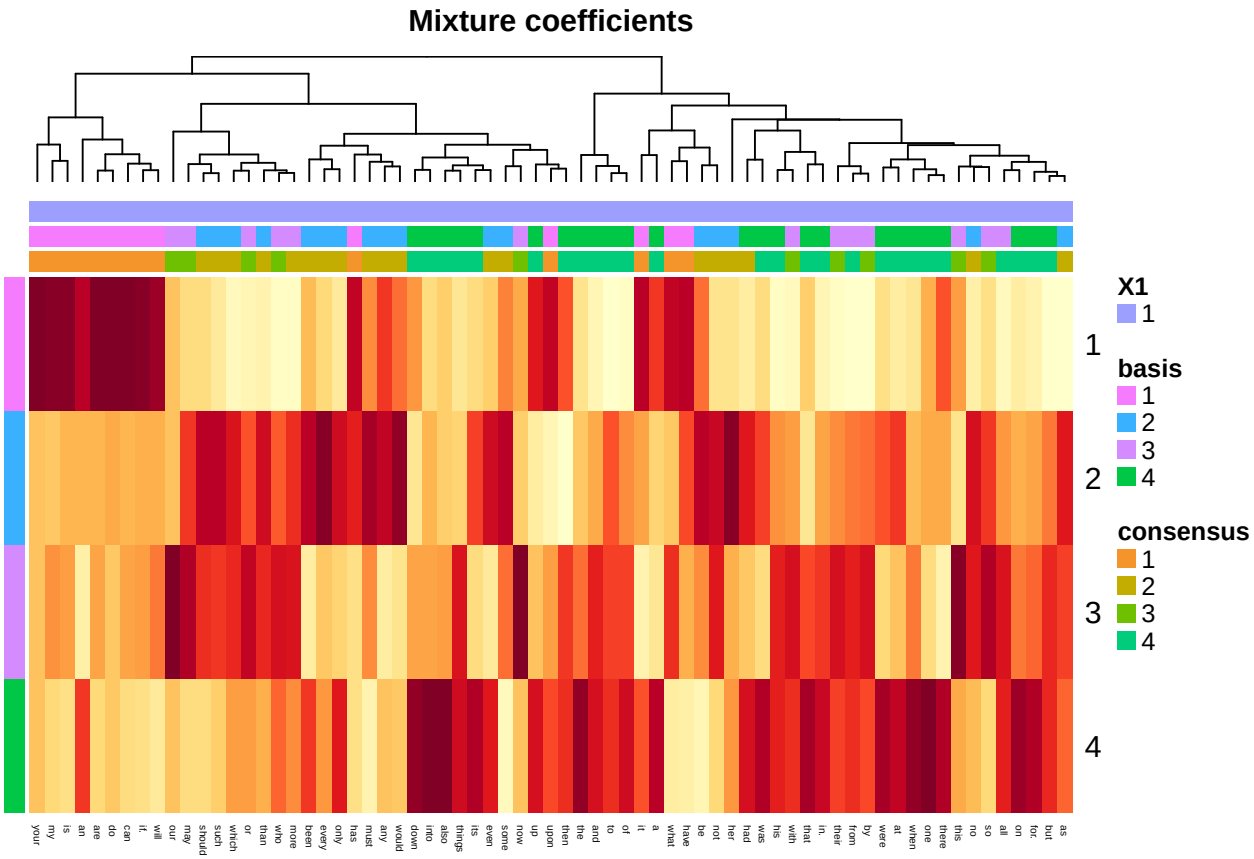

```
if (pkg_available("NMF")) {
  # Each row of H corresponds to one NMF factor.
  # These word lists are a starting point for interpretation.
  top_n <- 10
  top_words_by_factor <- apply(H, 1, function(v) {
    names(sort(v, decreasing = TRUE))[1:top_n]
  })
  top_words_by_factor
}
```

```
##       [,1]   [,2]  [,3]   [,4]
##  [1,] "my"   "her" "and"  "the"
##  [2,] "your" "to"  "the"  "and"
##  [3,] "is"   "of"  "our"  "was"
##  [4,] "do"   "and" "of"   "of"
##  [5,] "it"   "was" "to"   "to"
##  [6,] "are"  "had" "in."  "a"
##  [7,] "a"    "the" "his"  "in."
##  [8,] "will" "not" "with" "had"
##  [9,] "have" "be"  "not"  "his"
## [10,] "an"   "in." "that" "that"
```

```
if (pkg_available("NMF")) {
  # These plots can be helpful, but may be crowded depending on your screen size.
  basismap(nmf_fit, annRow = rownames(nmf_input), scale = "col", legend = FALSE)
  coefmap(nmf_fit, annCol = colnames(nmf_input), scale = "col", legend = FALSE)
```

```
}
```


Basis components
basis
1
2
3
4
X2
1
1
2
3
4




```
# Optional rank sensitivity scaffold.
# Try ranks 3, 4, and 5. How does your interpretation change?

ranks_to_try <- 3:5
nmf_rank_results <- lapply(ranks_to_try, function(r) {
  fit <- nmf(X_log, rank = r, nrun = 5, seed = 2026)
  W_r <- basis(fit)
  cl_r <- apply(W_r, 1, which.max)
  data.frame(
    rank = r,
    ari = if (pkg_available("mclust")) mclust::adjustedRandIndex(cl_r, TrueAuth) else NA_real_
  )
})

do.call(rbind, nmf_rank_results)
```

Problem 6 starter: wrap-up template

Use the following checklist to organize your final answer.

```
# 1. Best clustering method:
#    - Method:
#    - Preprocessing:
#    - Evidence:
#
# 2. Best visualization:
```

```
#    - Method:
#    - What it shows:
#    - What it misses:
#
# 3. Important words:
#    - Evidence from heatmap:
#    - Evidence from NMF:
#    - Evidence from PCA/MDS or other analyses:
#
# 4. Main lesson:
#    - What did you learn about clustering text count data?
```